Information Modeling Using EXPRESS — An Introduction

NB. NOTES ARE ON EVEN NUMBERED (LEFT HAND) PAGES AND THE PRESENTATION ON ODD NUMBERED (RIGHT HAND) PAGES.

Information?

This first part of the EXPRESS course covers the following topics:

- What is information, why do we need information models, and what is an information model?
- EXPRESS background
- STEP architecture
- TYPE
- ENTITY
- FUNCTION
- SCHEMA

We are basically concerned with storing and transmitting information in computer systems. In particular information about engineering data and processes.

Information?

Definition

Information is knowledge of ideas, facts and/or processes.

- Information can be exchanged between partners.
- Exchange may be real time or delayed.
- Information storage is a special case of communication.

Information Model?

Because we are dealing with computer systems we need a precise definition of the information of interest. One goal is to be able to exchange information between Computer Aided Design (CAD), Computer Aided Engineering (CAE), Computer Aided Manufacturing (CAM), etc., systems so they can interoperate.

The model may describe things in general, for example an animal has legs, and there can be data corresponding to the model about particular things — my cat Jessie has three legs (but she can still catch birds).

Information Model?

Definition

An information model is a formal description of types (classes) of ideas, facts and processes which together form a model of a portion of interest of the real world.

- If a model is written in EXPRESS or any other computer sensible representation, it has the additional property of being *computer processible*.
- An information model may be *instantiated* or *populated* to represent **particular** ideas, facts and processes.
- An information model that is specialized to take account of a particular instantiation method is a *concrete* model. One that is implementation independent is a *conceptual* model.

Why Formal Models?

In everday conversation, and writing, there can be many ambiguities and we are good at deciding which meaning is meant. Computers are notoriously bad at this.

We tend to use the surrounding context to help in disambiguation and if this is missing we have difficulties.

- The first two example sentences are not ambiguous (to us) but looking at the pair…
- The third sentence is ambiguous just by itself.
- Even a single word (braces) can be ambiguous.
- Numerical data (1-3-91) can be ambiguous.

There are some exercises about ambiguities.

Why Formal Models?

- Time flies like an arrow.
- Fruit flies like a banana.
- Joe saw the man with the telescope.
- Braces

  UK

  Hold up one’s trousers.

  USA

  Adjust one’s teeth.

- The date 1-3-91

UK

1st March 1991 (or is it 1891?)

USA

3rd January 1991

Information Model

This is a somewhat formal definition, but then we are going to be spending a lot of time with formal models and modeling.

It is intended to enable unambiguous communication about a limited topic (or range of topics).

Information Model

Purpose

To identify clearly the objects in a 'Universe of Discourse' in order to enable people (and potentially computer systems) to communicate precisely about a domain of common interest.

Comprises

An information model is composed of

- objects
- relationships
- constraints

which, taken together, provide a complete and unambiguous formal representation of a Universe of Discourse.

IM is NOT

We are interested in computer based/processible information models.

A model uses, or is associated with, various more common computer techniques, but it is essentially for human consumption.

IM is NOT

An Information Model is

- NOT a database definition (even though terms such as *schema* are common.
- NOT a data structure definition (even though data instances of the model could be structured)
- NOT a program (even though procedural code and algorithms may be in the model)

A populated instance of an IM may be maintained using DB or similar technologies. IM constraints

are often implemented via programatic code.

IM Description Methods

Historically, formal information models have been specified using either a written (lexical) language or using a graphical (drawings) language.

The graphic constructs are usually boxes and lines connecting the boxes, together with some annotations on the diagram.

A graphical model can easily be the size of a wall, which might cause difficulties if you ant to put one in a report.

IM Description Methods

An Information Model may be described:

Textually
: using a formally defined lexical language. Examples include EXPRESS, IISyCL (Integrated Information Systems Constraint Language), VDM (Vienna Development Method), etc.

Graphically
: using an iconic or diagramatic language such as EXPRESS-G, IDEF1X, OMT, UML, etc.

NOTE	Supplementing textual models with diagrams can help the reader's understanding. Graphical models nearly always require supplemental text for completeness.

EXPRESS Development

EXPRESS has been used, one way or another, for 20 years or so.

The requirement was for use in specifying industry and international standards.

Other modeling techniques were reviewed but did not have the power that was felt to be needed, in particular constraint specifications. Also the languages were basically graphical although there were some proprietry lexical adjuncts.

EXPRESS Development

EXPRESS developed as an information modeling language to meet the needs of product data exchange model definition.

- First version, called DSL, developed under the USAF funded PDDI program (early '80s).
- PDES reviewed NIAM and IDEF1X. Neither had the power needed.
- PDES started extending EXPRESS.

- STEP mandated all ‘Normative’ models to be in EXPRESS.
- Language still evolving.

REVIEWS

EXPRESS has been formally approved as an International Standard, specifically:

ISO 10303-11 *Industrial automation systems and integration — Product data representation and exchange — Part 11: Description method: The EXPRESS language reference manual*

The first edition was formally approved and published in 1994.

The second edition should be published during 2004.

REVIEWS

The language is subject to ongoing review within STEP and by other users. Also international public review as part of ISO standardization:

Early 1989
: ISO Draft Proposal ballot

Mid 1991
: ISO Committee Draft ballot

Oct 1991
: Ballot successful — Draft International Standard status.

Mid 1993
: Approved for registration as an International Standard (ISO 10303 Part 11).

End 1994
: Published as International Standard ISO 10303-11:1994.

End 2003
: Edition 2 approved as an International Standard.

Language Comparison

Most modeling anguages are graphical, which is inherently limiting.

For data modeling most languages are targeted towards Relational Databases. Examples include IDEF1X, Shlaer-Mellor, Extended Entity-Relation.

UML is for modeling an Object Oriented program. EXPRESS is for modeling data and naturally moved to an OO perspective (it was developed by practising engineers as user, not by computer scientists).

Table 1. LANGUAGE COMPARISON

Characteristic	Others	EXPRESS
Modeling		
Form	Graphics	Programmatic
Flavor	Relational	OO
Objects	X	X
Relationships	X	X
Attributes	X	X
Derived Atts.		X
Domain	Entity	Entity + Type
Sequencing		X
Cardinalities	Limited	Any
Constraints		
Domain	Limited	Any
Roles	Limited	Any
Categorization	Limited	Broad
Miscellaneous		
Multi-page	Some	X
Algorithms		X
Scoping		X

Graphical Models

Very good for group work — sketch on blackboard, but soon run out of space on the board. I have seen complete models that can take up a whole wall even with small print.

It’s difficult to check a model except by eyeballing it. It’s been a general experience over several decades of going from flowcharts to program code that many details get missed.

It is difficult to formally specify a graphical language.

Graphical Models

- Excellent for group explanations and work.
- Easy to follow (but can take a lot of wall space).
- Model development may be superficial (it looks right).
- Some drawing tools may exist, or can use CAD system.
- Effectively, not computer processible (What You See Is All You’ve Got).

Textual Models

Text languages for modeling can be formally defined, both syntax and much of the semantics. This means that they can be made computer processible and so can be automatically checked for correctness (syntax) and completeness.

They can represent a variety of modeling approaches, from mathematical or logical schemes to things more readily understood.

They can include a programming language so constraints can be expressed in terms of a process as well as in terms of rules and regulations.

They provide opportunities for models to be manipulated, for example automatically developing test cases or checking that data conforms to the model.

Textual Models

- Good formal definition or mathematical support.
- May be non-intuitive (e.g logic based methods).
- Complex constraints and rules.
- Computer processible.
- Syntax and semantic checking.
- Potential for automatic implementation (for model simulation and test).

EXPRESS is

NIAM and IDEF1X are both graphical languages for modeling Relational databases.

EXPRESS started as a single lexical language but has since expanded into a family of languages.

It was developed by a small group (about 4 at any given time) for modeling the kinds of information used in engineering. CAD models, Blueprints, Mechanisms, Engineering sign-off, and so on.

There were releases every quarter to a user group of about 50, who were full of their own suggestions and merrily changed the language in between times. In the first years there were no compilers (the language was changing too rapidly) so there were no technical constraints — every use of the language was perfect, no bugs, no complaints!

One of the strengths of EXPRESS is that it much of it was developed by the end users. That is also probaly its major weakness as its initial coherence sank under the weight.

EXPRESS is:

- A language family for representing an information model.

- Computer processible.
- Under development since early '80s.
- Superset of NIAM and IDEF1X representation capabilities.
- Exhibits an object oriented flavor.
- Been an ISO standard since 1994 (2nd Edition 2004)
- Has several aspects (subsets)

EXPRESS Aspects

The principal elements of EXPRESS are for representing things and the relationships between things (and as far as EXPRESS is concerned, a relationship is a thing). Groups of strongly related things can be collected together.

It includes a Pascal-like programming language for specifying complex constraints.

It is a conceptual moeling language, so puts no restrictions on the number of characters in a name, and arithmetic is infinitely precise.

There is a graphical form called EXPRESS-G which is a subset of the lexical language.

Another member of the family EXPRESS-I is a lexical language for displaying data that correspond to the concepts in EXPRESS.

Much more recently the third lexical language EXPRESS-X has been developed in which you can specify desired changes to an EXPRESS model and then have them performed; transformations principally consist of splitting or merging things and their relationships.

EXPRESS Aspects

- Textual language.
- Modeling of things and relationships (implementation independent).
- Algorithms for arbitrary constraint specifications.
- Modeling of implementation dependent data structures.
- Graphical form as a subset of textual form (EXPRESS-G).
- An ‘instantiation’ format (EXPRESS-I).
- Transformation specification (EXPRESS-X).

EXPRESS Usage

EXPRESS is widely used in the Standards community for formal definition of data-related concepts.

EXPRESS Usage

- Definition of the STEP models (200+ people from 20+ countries)
- Reverse engineering of a DBMS system
- Software Specification Document for a CAD geometry processor
- Electronic standards (VHDL, EDIF, CFI etc)
- Many European ESPRIT projects
- Data Definition Language for OO Database
- Geological modeling
- Genome modeling

Other uses are possible, such as using EXPRESS to define the syntax, grammer, and semantics of the EXPRESS language.

STEP History

The story starts in the mid 1970's with a small group trying to develop an ANSI standard for geometry data. At the end of the 70's McAuto (part of McDonnel Douglas) got a contract from CAM-I (Computer Aided Manufacturing — International) to develop a standard for data exchange between solid modeling systems; the result was not well received.

Just after this Boeing (Walt Braithwaite), GE (Phil Kennicott) and the then National Bureau of Standards (Roger Nagel) produced IGES — Initial Graphics Exchange Specification for data exchange between CAD (Computer Aided Drawing) systems. This was reluctantly implemented by the major CAD vendors and rapidly became the ANSI Y14.6M standard (the last section of which was the McAuto work). Then came a proliferation of standards.

As IGES was not written in France the French published their SET standard. CAM-I still wanted a solid model data exchange mechanism and came up with the XBF (Experimental Boundary File), an extension of IGES, which itself was going through several expansions. The Germans produced VDAFS specifically for sculptured surfaces as used for car bodies. The XBF work moved under the IGES umbrella and became ESP (Experimental Solids Proposal).

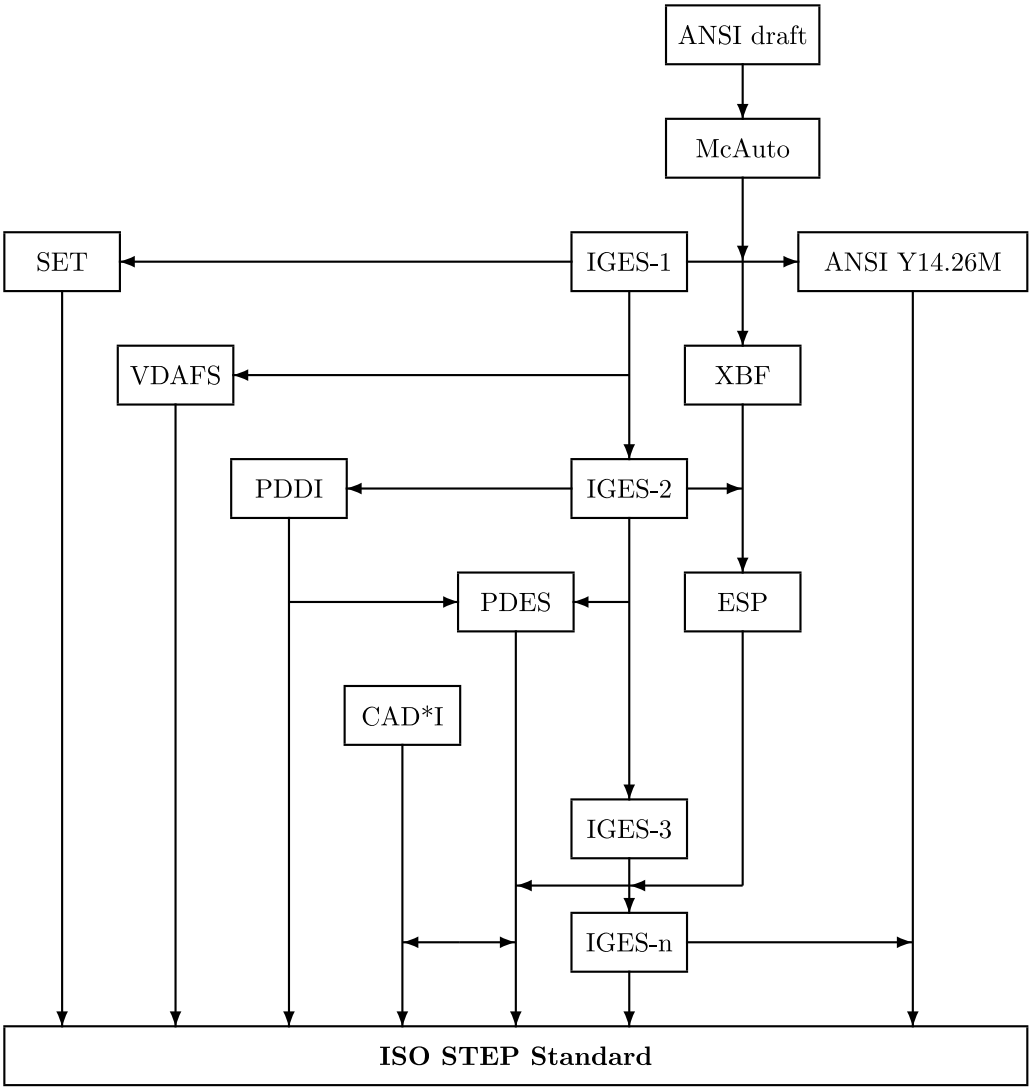
The USAF gave McDonnell Douglas a 2 part contract to (a) for a small amount of money determine if IGES met USAF (and industry) requiremnts and if the did not (b) for a large amount of money develop something that did. Unsurprisingly they determined that IGES was unsuitable and so came up with the PDDI standard. There was also yet another effort going on in Europe called the CAD*I project funded under the ESPRIT program.

IGES was experiencing growing pains and it seemed sensible to make a fresh start. Boeing (Kal Brauner and Dave Briggs) proposed PDES — Product Data Exchange Standard based on the best work from the US. In particular they strongly urged that it should have a formal basis.

Somehow the international community got together and demanded just one standard — STEP, Standard for the Exchange of Product Model Data, to be based on the technical work from the PDES

group.

After a while some countries got upset as they felt that it had become a US standard (even though most participants were non-US). This dilemma was eventually resolved by changing PDES to be — Product Data Exchange using STEP (which some then called Standard for Exchange using PDES).



STEP Documents

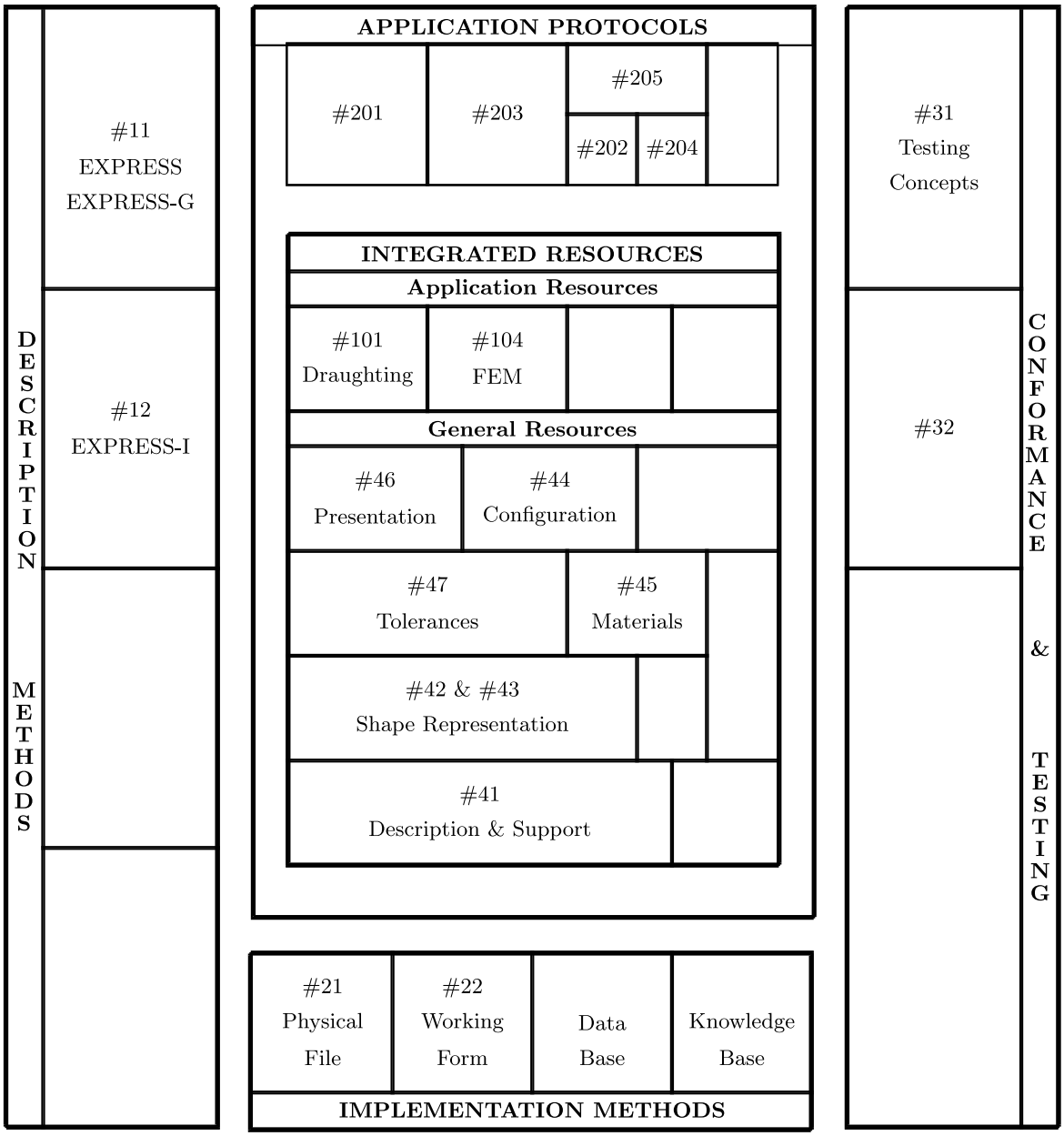
The STEP standard, ISO 10303, is really a suite of cooperating standards each member of which is a *Part* of ISO 10303.

The Parts are grouped into *series*.

- Parts in the range 11-19 form the *Description Methods* series, which include the EXPRESS family.
- Parts in the range 21-29 form the *Implemantation Methods* series defining how to exchange data that corresponds to an EXPRESS model.
- Parts in the range 31-39 form the *Conformance and Testing* series defining how to test STEP implementations.
- Parts in the range 41-99 form the *Resources* series which define an integrated set of application

independent EXPRESS information models for product descriptions.

- Parts in the range 201+ form the *Application Protocol* (AP) series which specify application dependent information models for the purposes of data exchange.



STEP Architecture

The STEP architecture is centered around the Integrated Resource Models (IRs), which are defined using EXPRESS.

An Application Protocol (AP) is a subset of the IRs. It includes an EXPRESS model mapped from the EXPRESS models in the IRs.

The implementation methods, called Level 1, Level 2, and so on, are exchange mechanisms for data that corresponds to an EXPRESS model. They essentially consist of a mapping from EXPRESS to a data representation.

As far as a typical end user is concerned, the IRs are invisible and there are APs and exchange

levels.

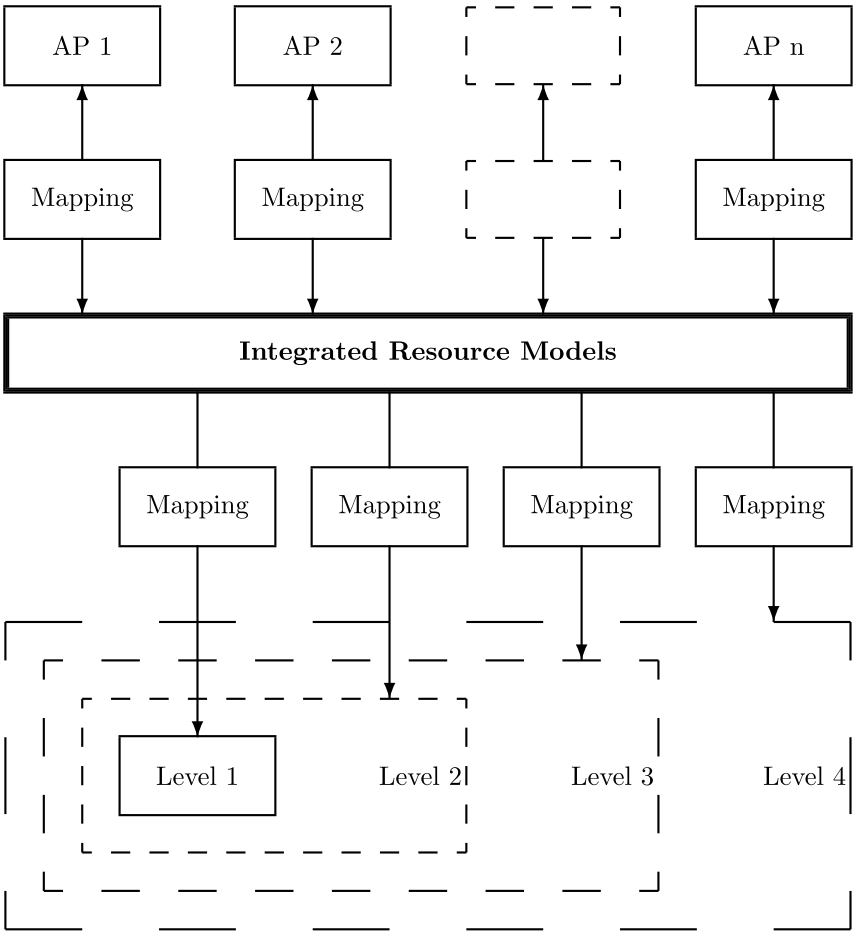


Figure 1. STEP ARCHITECTURE

Level 1 Exchange

Level 1 data exchange is file-based. Get your CAD system to create a STEP data file then archive it and/or send it to someone else (to read into their CAD system).

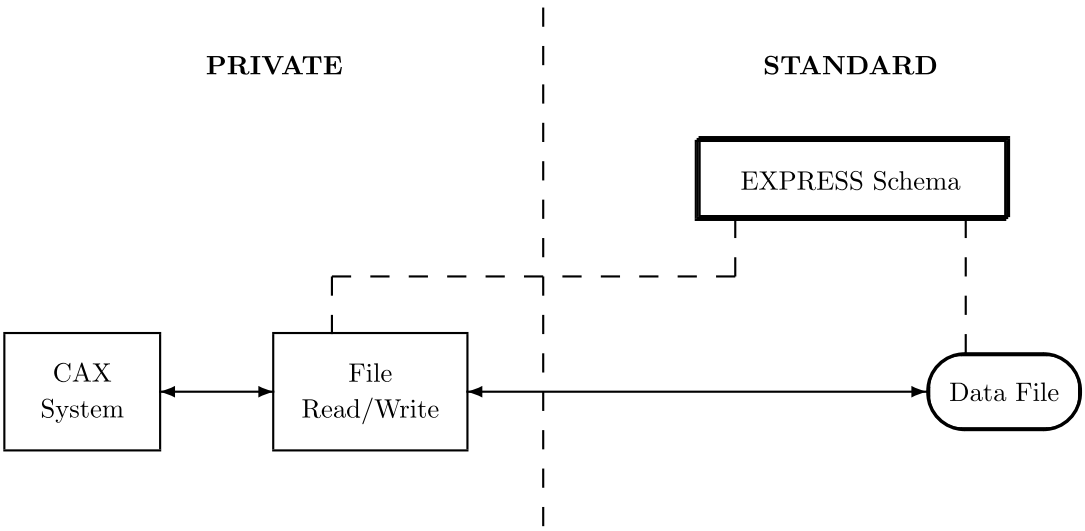


Figure 2. Level 1 Exchange

Level 2 Exchange

Level 2 data exchange is memory-based. Get your CAD system to create a (temporary) STEP database which you can then query and change. The data can be written to a file for Level 1 use. At the end of the session the STEP database is no longer available.

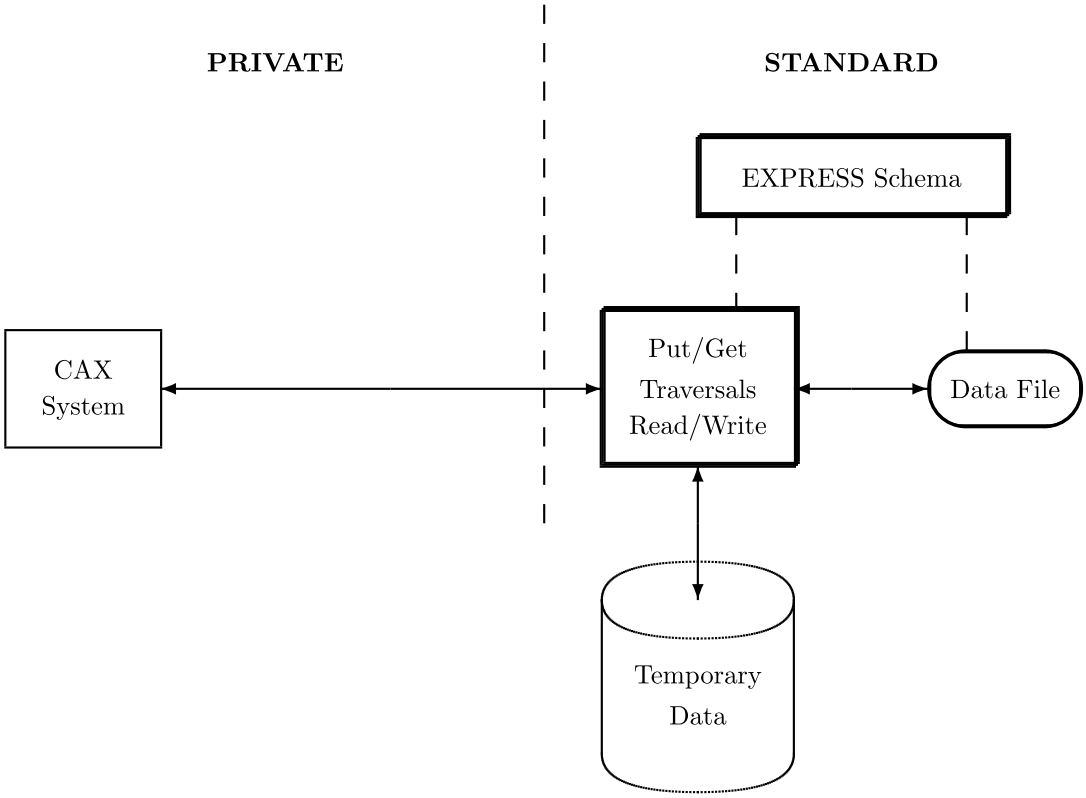


Figure 3. Level 2 Exchange

Level 3 Exchange

Level 3 data exchange is database-based. The STEP data is maintained in a (permanent) shared database. STEP level 1 files can be written and read by the database.

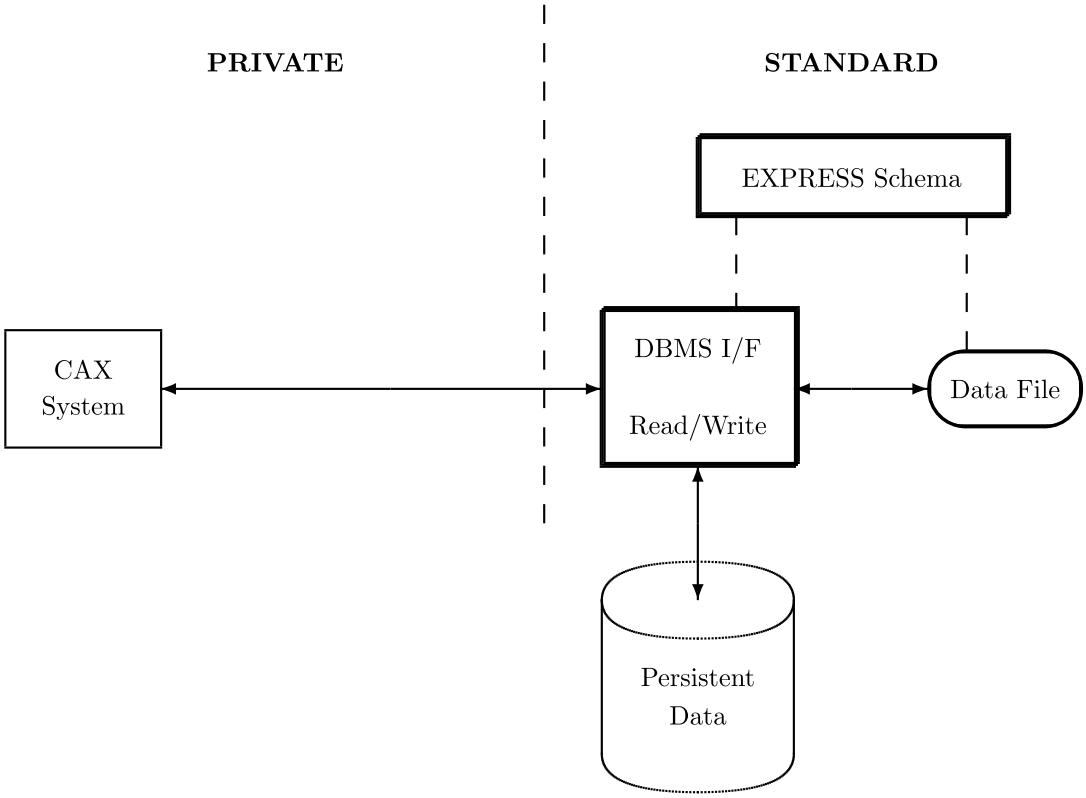


Figure 4. LEVEL 3 EXCHANGE

Procedural Exchange

This allows not only data, but also commands (and their results) to be passed into and out of a CAX program in a standardised manner.

For example, instead of inserting the data representing, say, a block with a hole in it, tell the system to create a block, put a hole in it, and then perhaps move it to another position. The end result in terms of data values can be the same but the route is very different.

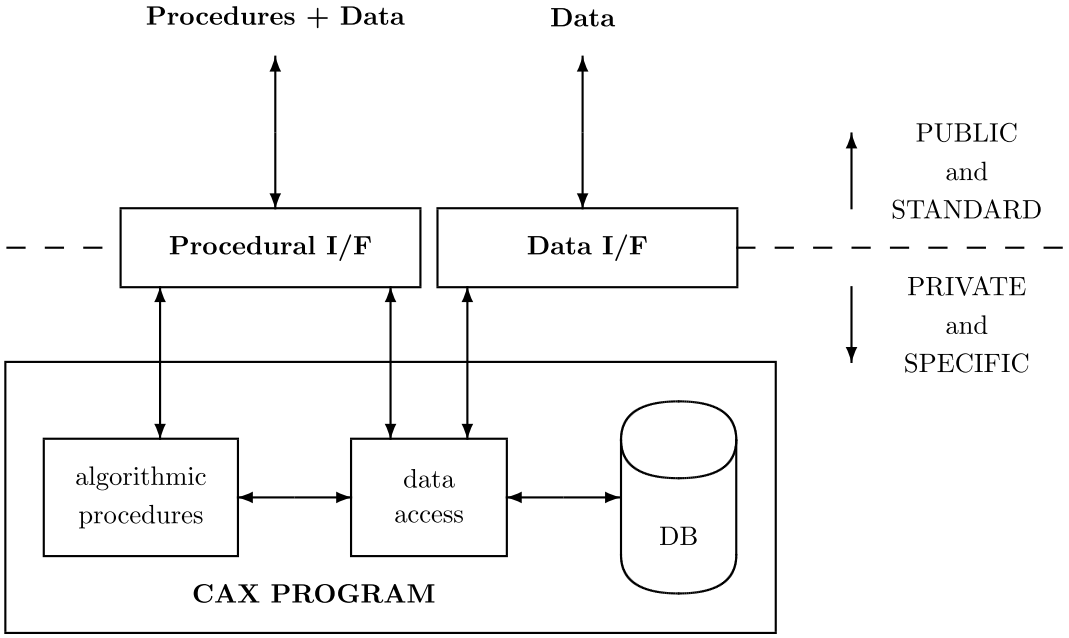


Figure 5. Procedural Exchange

Level 4 Exchange

This was the vision when STEP started — intelligent knowledgebases as an exchange mechanism.

The vision has faded.

The majority of STEP implementations are Level 1 (file exchange). Internally, though, they are implemented using a Level 2 or 3 architecture.

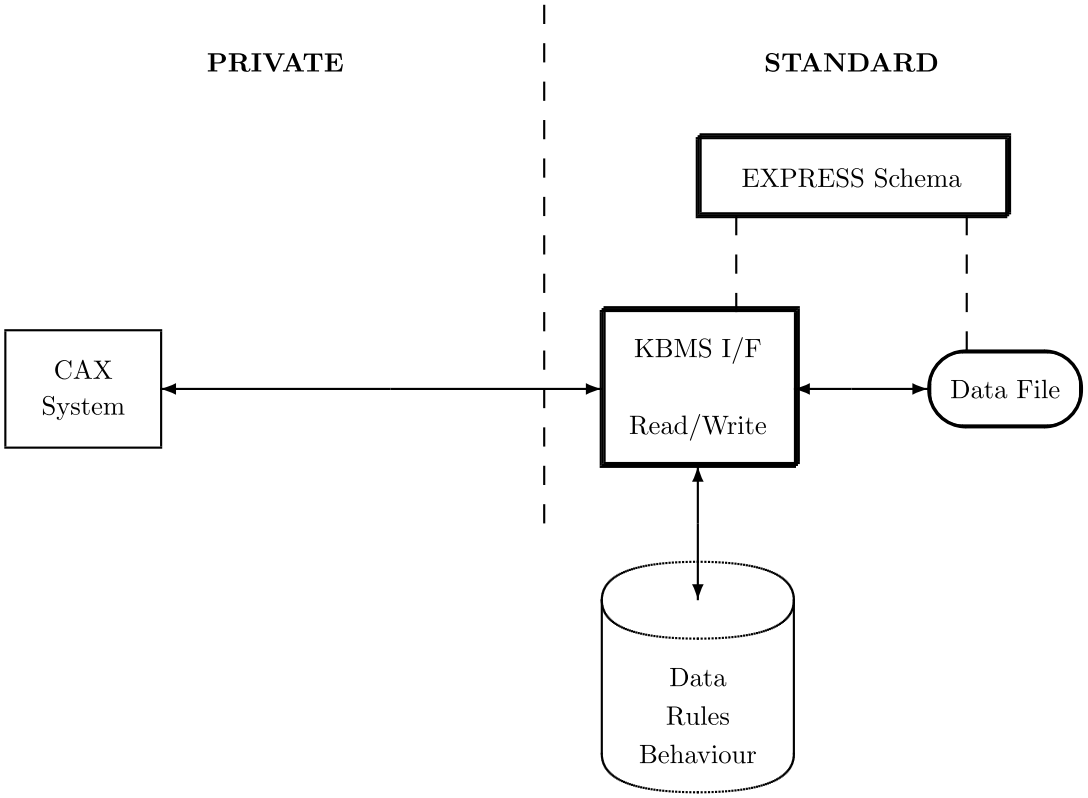


Figure 6. LEVEL 4 EXCHANGE

EXPRESS Primitives

These, plus literals, are the fundamental ‘things’ of the EXPRESS language.

- Numbers, etc., are the most elementary
- Schema, etc., are the most complex
- Aggregations are collections of things
- The procedural language is an imperitive programming language.

These are later described in detail.

EXPRESS Primitives

- Number, Integer, Real, Binary, String, Boolean (T/F), Logical (T/F/U)
- Schema, Entity, Rule, Function, Procedure, Type (Defined, Select, Enumeration)

- Aggregations — Array, Set, List, Bag
- Pascal-like procedural language

Simple Types

- NUMBER is any kind of number with any value.
- REAL is a decimal kind of NUMBER.
- INTEGER is an integer kind of NUMBER and is a kind of REAL number.

The numbers have infinite precision and can be as large or small as you like.

The procedural language lets you perform operations on NUMBERs.

Simple Types

- `n : NUMBER` which has 'subtypes'
  - `i : INTEGER`
  - `r : REAL`

These types may be given a 'precision'. E.g `REAL(6)`

Various operations such as `+, -, //, ">="`, etc. may be applied to these types.

Simple Types (cont)

EXPRESS provides for both 2- and 3-valued logical statements and epressions.

The procedural language lets you perform operations on logicals.

Simple Types (cont)

- `l : LOGICAL` has values `FALSE`, `UNKNOWN`, and `TRUE`, with `FALSE < UNKNOWN < TRUE`.
- `b : BOOLEAN` is a 'subtype' of `LOGICAL` having values of `FALSE` and `TRUE` only.

Comparisons on Booleans and Logicals can be performed (e.g `=, <, "<=", "<>"`, etc.)

Other operations include `NOT`, `AND`, `OR`, `XOR`.

Simple Types (cont)

A STRING is any sequence of any number of characters. A BINARY is a specialisation of a STRING as it is limited to the digits 0 and 1.

The procedural language lets you perform operations (concatenation, subsetting and comparison) on strings.

Simple Types (cont)

- `s : STRING` - a sequence of characters
- `bin : BINARY` - a sequence of bits (0s and 1s)

These may be dynamic or fixed with a maximum size. For example `STRING(6) FIXED`.

These types may be concatenated and compared, and subsets addressed via indexing. For example

```
s1 : STRING := 's';
s2 : STRING := 'its';
.....
s1 := s1 + s2;
IF s1[2:3] = 'it' THEN ...
```

Aggregations

Aggregations are collections of things. A collection may be ordered or unordered, and fixed or expandible in size, and with or without duplicates.

Aggregations

General form is `AGGR [L:H] OF ⋯` where L and H are the Low and High bounds respectively (`H >= L`), and containing N elements. Bags, Lists and Sets may have an indefinite high bound denoted by '?' character.

ARRAY

Ordered collection of elements. `N = (H-L+1)`.

BAG

Unordered collection with possibly duplicate elements. `L <= N <= H " where " L >= 0`.

LIST

Ordered collection with possibly duplicate elements. `L <= N <= H " where " L >= 0`.

SET

Unordered collection with no duplicate elements. `L <= N <= H " where " L >= 0`.

> **NOTE** `LIST [L:H] OF UNIQUE ⋯` is used for an ordered collection with no duplicates.

Types

A TYPE is a user-defined extension to the EXPRESS-defined simple types and aggregations. Every TYPE has a name chosen by the user.

Types

User defined extensions to the simple types and aggregations.

Defined

A 'renaming' of a simple type or aggregation.
`TYPE volume = REAL; END_TYPE;`

Select

A selection among some types.
`TYPE choose = SELECT(a,b,c); END_TYPE;`

Enumeration

An ordered set of values represented by names.
`TYPE enum = ENUMERATION OF (up, down);`
`END_TYPE;`

TYPE Examples

`tt "things"` illustrates an aggegration of an aggregation.

`tt "gender"` is an ENUMERATION because the possiblities (except for some pathological cases) are known.

`tt "hair_type"` is not a particularly good example, but it does imply a limited scope for the model.

`tt "choose_thing"` is a selection between two alternatives.

TYPE Examples

```
TYPE things = SET [1:?] OF
              LIST [1:?] OF thing;
END_TYPE;

TYPE date = ARRAY [1:3] OF INTEGER;
END_TYPE;

TYPE gender = ENUMERATION OF
              (male, female);
END_TYPE;
```

```
TYPE hair_type = ENUMERATION OF
                 (blonde, black, bald);
END_TYPE;

TYPE choose_thing = SELECT
                    (thing1, thing2);
END_TYPE;
```

ENTITY

An ENTITY is a user defined object, representing some thing. It has various components which will be described. Every ENTITY has a user-defined name.

ENTITY

An entity represents an object of interest in the model of the Universe of Discourse.

The characteristics (properties) of an entity are defined in terms of data (attributes) and behaviour (constraints).

An entity may ‘inherit’ properties from another entity.

ENTITY Attributes

An attribute is some kind of data element that helps characterize the ENTITY. An attribute consists of a user-defined name and a specification of the kind of data.

The kind of data may be a (collection of) simple types, TYPEs or ENTITYs.

ENTITY Attributes

Attributes are either *explicit* or *derived*.

```
ENTITY circle;
  center : point;
  radius : length;
DERIVE
  perimeter : length := 2.0*PI*radius;
END_ENTITY;

TYPE length = REAL; END_TYPE;
```

The data for calculating a derived attribute must be accessible from the entity.

ENTITY Constraints

Constraints limit the kind and/or values of the attributes' data.

UNIQUE In this case no two circles can have the same center AND radius.

WHERE rules are logical expressions. In this case the radius must be positive length.

ENTITY Constraints

Attribute values within entity instances may be constrained by either uniqueness requirements or by domain rules (WHERE clauses). These apply to *every* instance of the entity.

```
ENTITY circle;
  center : point;
  radius : length;
UNIQUE
  un1 : center, radius;
WHERE
  pos_rad : radius > 0.0;
END_ENTITY;
```

A WHERE (domain) rule fails if it evaluates to `FALSE`.

Example ENTITY

The attributes are those things of interest about a person.

Not everyone has a nickname.

Not everyone has a spouse.

No two people have the same social security number.

The WHERE rule states that if someone has a spouse then the spouse must be of the opposite sex.

Example ENTITY

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  nickname   : OPTIONAL STRING;
  ss_no      : INTEGER;
  sex        : gender;
  spouse     : OPTIONAL person;
  children   : SET [0:?] OF person;
```

```
UNIQUE
  un1 : ss_no;
WHERE
  w1 : (EXISTS(spouse) AND sex <> spouse.sex)
       OR NOT EXISTS(spouse);
END_ENTITY;
```

Subtyping

A Subtype is a special kind of its supertype(s).

Forgetting about Cantor and degrees of infinity

- There are fewer odd numbers than there are natural numbers.
- There are fewer prime numbers than there are natural numbers.

Subtyping

Subtypes inherit ther properties of their Supertypes.

```
ENTITY natural_number;
  value : INTEGER;
END_ENTITY;

ENTITY odd_number
  SUBTYPE OF (natural_number);
  ...
END_ENTITY;

ENTITY prime_number
  SUBTYPE OF (natural_number);
  ...
END_ENTITY;
```

FUNCTION Example

These are part of EXPRESS programming language aspects.

The particular example takes two aggregations and returns either TRUE or FALSE depending on whether or not the first is a subset of the second (i.e., every member of `tt "sub"` is also in `tt "super"`).

FUNCTION Example

Used for constraint definition and for derived attributes.

```
FUNCTION subset(sub,super :
         AGGREGATE OF GENERIC) : BOOLEAN;

  IF (SIZEOF(sub) > SIZEOF(super)) THEN
    RETURN(FALSE);
  END_IF;
  REPEAT i := 1 TO SIZEOF(sub);
    IF (sub[i] IN super) THEN
      super := super - sub[i];
    ELSE
      RETURN(FALSE);
    END_IF;
  END_REPEAT;
  RETURN(TRUE);

END_FUNCTION;
```

Predefined Functions

EXPRESS includes a variety of predefined functions.

There is more on these later in the course.

Predefined Functions

- Mathematical (e.g ABS, SIN, SQRT etc)
- Aggregation sizes (e.g LOBOUND, HIBOUND, SIZEOF, LENGTH)
- Number/String conversion (FORMAT, VALUE)
- EXISTS(V) checks for existance of OPTIONAL attribute V.
- NVL(ATTR; SUBS) if ATTR has a value, then ATTR is returned, else SUBS is returned.
- TYPEOF(V) returns the set of types of V.
- USEDIN(T; R) takes an entity T and its role R that it plays in other entities and returns each entity instance that uses T in role R.

Constants

EXPRESS includes the mathematical constants `Pi` and `e` (to infinite precision).

You can also define your own constants, but this is not often done.

Constants

- Some predefined constants (PI, e).

- User-defined constants

```
CONSTANT
  thousand : NUMBER := 1000;
  million  : NUMBER := thousand**2;
  origin   : point := point(0.0, 0.0);
END_CONSTANT;
```

SCHEMA

The minimum EXPRESS model consists of a single empty SCHEMA.

TYPE, ENTITY, FUNCTION definitions are contained within a SCHEMA.

SCHEMA

- A SCHEMA contains the objects, relationships and constraints for a particular domain of interest.
- Schemas provide a mechanism for partitioning the ‘real world’ into relevant domains.
- There must be well defined limits to the domain represented via a Schema --- a single Schema should not be used to describe two different domains of interest.

SCHEMA (cont)

A model usually consists of more than one SCHEMA.

From within a SCHEMA you can get at anything in any other SCHEMA (there is no way to ‘hide’ something).

SCHEMA (cont)

- An EXPRESS model may contain more than one Schema.
- Where multiple Schemas are used there is normally one ‘main’ schema and n ‘subsidiary’ schemas.

```
SCHEMA main;
  REFERENCE FROM sub1 ...
  -- types, entities, rules, etc.
END_SCHEMA;

SCHEMA sub1;
  -- types, entities, rules, etc.
END_SCHEMA;
```